

บทที่ 14

Lab: Traffic Shaping ด้วย Token Bucket และ Leaky Bucket

14.1 วัตถุประสงค์

Learning Objectives

เมื่อจบ Lab นี้ ผู้เรียนจะสามารถ:

- อธิบายหลักการทำงานของ Token Bucket และ Leaky Bucket algorithm
- จำลอง network topology และจำกัด bandwidth ด้วย Mininet
- สร้าง traffic แบบ bursty ด้วย Scapy เพื่อทดสอบ shaping
- ใช้ tc (Traffic Control) ตั้งค่า Token Bucket Filter (TBF) บน Linux
- วิเคราะห์ผลลัพธ์ด้วย Wireshark/tcpdump และเปรียบเทียบ throughput, delay, packet loss ก่อน-หลัง shaping
- เปรียบเทียบพฤติกรรมของ Token Bucket กับ Leaky Bucket ภายใต้ input pattern เดียวกัน

14.2 ความรู้พื้นฐานที่เกี่ยวข้อง

ปัญหาของ Bursty Traffic

เครือข่ายที่ไม่มีการควบคุม traffic อาจเกิด **burst** ขนาดใหญ่ในเวลา สั้น ๆ ทำให้ router/switch เกิด **congestion**, packet loss และ delay ที่ไม่แน่นอน (jitter) การ **Traffic Shaping** คือการควบคุมอัตราการส่งข้อมูลให้สม่ำเสมอ (smooth) ก่อนเข้าสู่ เครือข่าย

Leaky Bucket Algorithm

หลักการ

เปรียบเหมือนถังที่มีรูรั่วด้านล่าง:

- Packet ที่เข้ามาจะถูกเก็บไว้ใน **buffer/queue** (ขนาดคงที่)
- ข้อมูลจะถูกปล่อยออกด้วย **อัตราคงที่** (constant rate) ไม่ว่า input จะ burst แค่ไหน
- หาก buffer เต็ม packet ที่เข้ามาใหม่จะถูก **drop**

ผลลัพธ์: output traffic จะราบเรียบเสมอ (perfectly smoothed)

Token Bucket Algorithm

หลักการ

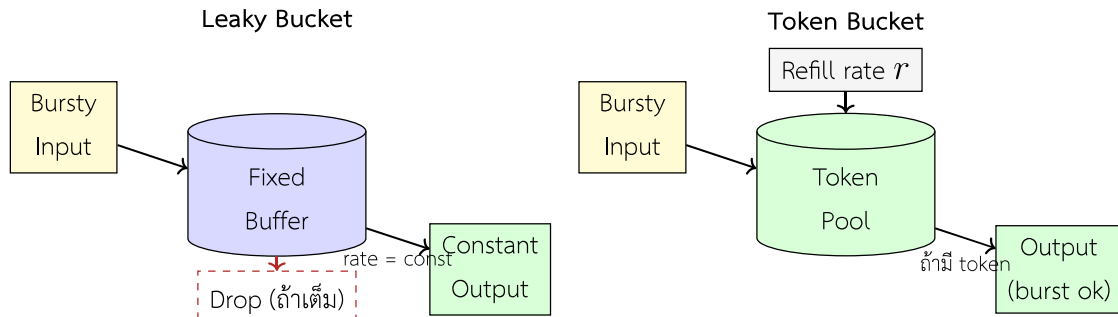
เปรียบเหมือนถังเก็บ **token** (สิทธิ์ในการส่งข้อมูล):

- Token จะถูกเติมเข้าถังด้วยอัตราคงที่ r (token/sec)
- ถังมีขนาดจำกัด b (burst size) — token ส่วนเกินจะถูกทิ้ง
- Packet จะถูกส่งได้ก็ต่อเมื่อมี token เพียงพอ (1 token = 1 byte หรือ 1 packet ตามการออกแบบ)
- หากถังมี token สะสมอยู่ (ไม่ได้ส่งข้อมูลมาสักพัก) packet สามารถส่งแบบ **burst** ได้จนกว่า token จะหมด

ผลลัพธ์: อนุญาตให้เกิด burst ได้ในขอบเขตที่กำหนด (burst-tolerant), ต่างจาก Leaky Bucket ที่บีบให้ราบเรียบเสมอ

คุณสมบัติ	Leaky Bucket	Token Bucket
Output rate	คงที่เสมอ	คงที่โดยเฉลี่ย, burst ได้
รองรับ burst	ไม่รองรับ	รองรับ (จำกัดด้วย bucket size)
การ implement	queue + fixed drain rate	counter + refill rate
เหมาะกับ	Traffic ที่ต้อง smooth มาก	Traffic ที่มี burst เป็นปกติ

ตารางที่ 14.1: เปรียบเทียบ Leaky Bucket และ Token Bucket



รูปที่ 14.1: เปรียบเทียบโครงสร้าง Leaky Bucket และ Token Bucket

Token Bucket Filter (TBF) บน Linux

Linux มี queueing discipline (qdisc) ชื่อ **TBF** ที่ implement Token Bucket algorithm ให้ใช้งานผ่านคำสั่ง tc โดยตรง โดยมี parameter หลัก 3 ตัว:

Parameter	ความหมาย
rate	อัตราการปล่อย token (เทียบเท่า sustained bandwidth)
burst	ขนาดของ bucket (byte) — ปริมาณ burst สูงสุดที่ยอมรับ
latency	เวลาสูงสุดที่ packet รอได้ก่อนถูก drop

ตารางที่ 14.2: Parameter หลักของ TBF

14.3 การกำหนดค่า Parameter ประจำกลุ่ม

เหตุผลที่แต่ละกลุ่มได้ค่าต่างกัน

เพื่อให้แต่ละกลุ่มได้ผลการทดลองที่ **แตกต่างกัน** ป้องกันการลอกผลลัพธ์ระหว่างกลุ่ม และฝึกการวิเคราะห์ trade-off ที่หลากหลาย ค่า parameter จะถูก generate จาก **รหัสกลุ่ม** ของ นักศึกษาแต่ละกลุ่ม โดยใช้ script ด้านล่าง

ตัวอย่างโค้ด 14.1: generate_group_params.py

```

1  #!/usr/bin/env python3
2  import hashlib
3  import random
4  import sys
5
6  def generate_group_params(group_id: str):
7      seed = int(hashlib.md5(group_id.encode()).hexdigest(), 16) % (2**32)
8      rng = random.Random(seed)
9

```

```

10     params = {
11         "group_id": group_id,
12         "bandwidth_mbps": rng.choice([2, 5, 8, 10, 15]),
13         "token_rate_kbit": rng.choice([256, 512, 1024, 2048]),
14         "bucket_size_kbit": rng.choice([16, 32, 64, 128]),
15         "burst_size_packets": rng.randint(20, 100),
16         "burst_interval_ms": rng.choice([50, 100, 200, 500]),
17         "packet_size_bytes": rng.choice([512, 1024, 1400]),
18         "algorithm": rng.choice(["token_bucket", "leaky_bucket"]),
19         "latency_ms": rng.choice([50, 100, 200, 400]),
20     }
21     return params
22
23 if __name__ == "__main__":
24     group_id = sys.argv[1] if len(sys.argv) > 1 else "G00"
25     p = generate_group_params(group_id)
26     print(f"=== Parameter สำหรับกลุ่ม {p['group_id']} ===")
27     for k, v in p.items():
28         if k != "group_id":
29             print(f" {k}: {v}")

```

วิธีใช้งาน

```
$ python3 generate_group_params.py G07
```

```
=== Parameter สำหรับกลุ่ม G07 ===
```

```

bandwidth_mbps: 8
token_rate_kbit: 1024
bucket_size_kbit: 32
burst_size_packets: 67
burst_interval_ms: 100
packet_size_bytes: 1024
algorithm: token_bucket
latency_ms: 200

```

ให้แต่ละกลุ่มรัน script นี้ก่อนเริ่ม Lab และบันทึกค่าที่ได้ลงในตาราง บันทึกผล (ท้ายบท) เพื่อใช้ตลอดการทดลอง

14.4 อุปกรณ์และการเตรียมความพร้อม

องค์ประกอบ	รายละเอียด
Mininet	จำลอง topology h1 -- s1 -- h2
Scapy	สร้าง bursty traffic ตาม parameter ของกลุ่ม
tc (iproute2)	ตั้งค่า TBF qdisc บน interface
Wireshark/tcpdump	Capture และวิเคราะห์ traffic ก่อน-หลัง shaping

ตารางที่ 14.3: อุปกรณ์ที่ใช้ใน Lab

ตัวอย่างโค้ด 14.2: ติดตั้ง Mininet และเครื่องมือที่จำเป็น

```
$ sudo apt update
$ sudo apt install -y mininet python3-scapy tcpdump wireshark iproute2
```

14.5 ส่วนที่ A: สร้าง Topology บน Mininet

ตัวอย่างโค้ด 14.3: topo_bucket.py

```
1  #!/usr/bin/env python3
2  from mininet.net import Mininet
3  from mininet.node import OVSSwitch
4  from mininet.cli import CLI
5  from mininet.link import TCLink
6  from mininet.log import setLogLevel
7
8  def build_topology(bandwidth_mbps):
9      net = Mininet(switch=OVSSwitch, link=TCLink)
10
11     h1 = net.addHost('h1', ip='10.0.0.1/24')
12     h2 = net.addHost('h2', ip='10.0.0.2/24')
13     s1 = net.addSwitch('s1')
14
15     # จำกัด bandwidth ที่ link ตามค่าประจำกลุ่ม
16     net.addLink(h1, s1, bw=bandwidth_mbps)
17     net.addLink(h2, s1, bw=bandwidth_mbps)
18
19     net.start()
20     return net
21
22 if __name__ == '__main__':
```

```

23     setLogLevel('info')
24     BANDWIDTH = 8# (*@ \thaifont\small \textcolor{themeredแทนที่ด้วยค่า}{ bandwidth\_mbps ของกลุ่มตนเอง}@*)
25     net = build_topology(BANDWIDTH)
26     CL(net)
27     net.stop()

```

ตัวอย่างโค้ด 14.4: รัน topology

```

$ sudo python3 topo_bucket.py
mininet> h1 ping -c3 h2
mininet> xterm h1 h2

```

14.6 ส่วนที่ B: ตั้งค่า Token Bucket Filter (TBF)

ตัวอย่างโค้ด 14.5: ตั้งค่า TBF บน interface ของ h1

```

# ภายใน xterm ของ h1
h1# tc qdisc add dev h1-eth0 root tbf \
    rate 1024kbit \
    burst 32kbit \
    latency 200ms

# ตรวจสอบค่าที่ตั้ง
h1# tc -s qdisc show dev h1-eth0

```

ข้อสังเกต

ค่า rate และ burst ต้องแทนที่ด้วยค่า token_rate_kbit และ bucket_size_kbit ที่ได้จาก script generate parameter ของกลุ่มตนเอง

ตัวอย่างโค้ด 14.6: จำลอง Leaky Bucket ด้วย tc (คิว + drain คงที่)

```

# Leaky Bucket เทียบเท่ากับ TBF ที่มี burst โกล่เคียง 0
# (แทบไม่ยอมให้เกิด burst เลย)
h1# tc qdisc add dev h1-eth0 root tbf \
    rate 1024kbit \
    burst 1600 burst ขนาดเล็กมาก 1 packet \
    latency 200ms

```

ข้อจำกัดของการจำลองด้วย tc

Linux TBF โดยธรรมชาติคือ Token Bucket การจำลอง Leaky Bucket จึงเป็น เพียง การประมาณ โดยลด burst size ให้เล็กที่สุด เพื่อบีบไม่ให้เกิด burst หากต้องการ Leaky Bucket ที่แท้จริง (strict constant-rate output) ต้อง implement ด้วย script เอง (ดู Part D)

14.7 ส่วนที่ C: สร้าง Bursty Traffic ด้วย Scapy

ตัวอย่างโค้ด 14.7: generate_burst_traffic.py

```

1  #!/usr/bin/env python3
2  from scapy.all import IP, UDP, Raw, send
3  import time
4  import sys
5
6  def send_burst(dst_ip, burst_size, packet_size, interval_ms, num_bursts=10):
7      (*@ \thaifont\small ""ส่ง traffic แบบburst ตามparameter ของกลุ่ม"" @*)
8      payload = 'X' * packet_size
9      for burst_num in range(num_bursts):
10         print(f'[Burst {burst_num+1}] ส่ง {burst_size} packets')
11         for _ in range(burst_size):
12             pkt = IP(dst=dst_ip) / UDP(dport=5000) / Raw(load=payload)
13             send(pkt, verbose=0)
14             time.sleep(interval_ms / 1000)
15
16  if __name__ == '__main__':
17      DST_IP = "10.0.0.2"
18      BURST_SIZE = 67# (*@\textcolor{themered}{burst\_size\_packets} ของกลุ่ม"@*)
19      PACKET_SIZE = 1024# (*@\textcolor{themered}{packet\_size\_bytes} ของกลุ่ม"@*)
20      INTERVAL_MS = 100# (*@\textcolor{themered}{burst\_interval\_ms} ของกลุ่ม"@*)
21
22      send_burst(DST_IP, BURST_SIZE, PACKET_SIZE, INTERVAL_MS)

```

แบบฝึกหัด C

1. รัน tcpdump บน h2 เพื่อดักจับ traffic ก่อน apply TBF
2. รัน generate_burst_traffic.py จาก h1 ไปยัง h2
3. บันทึกไฟล์ capture เป็น before_shaping.pcap
4. Apply TBF ตาม Part B แล้วรัน traffic ซ้ำอีกครั้ง
5. บันทึกไฟล์ capture เป็น after_shaping.pcap

ตัวอย่างโค้ด 14.8: Capture traffic ทั้งสองฝั่ง

```

# บน h2 (ก่อน apply TBF)
h2# tcpdump -i h2-eth0 -w before_shaping.pcap &

# บน h1 (รัน traffic generator)

```

```
h1# python3 generate_burst_traffic.py
```

```
# หยุด capture แล้วทำซ้ำหลัง apply TBF บน h1
h2# kill %1
```

14.8 ส่วนที่ D: (Optional) Implement Leaky Bucket ด้วย Python

สำหรับกลุ่มที่ได้ algorithm: leaky_bucket ให้ implement algorithm เองเพื่อเปรียบเทียบกับ TBF ของ Linux โดยตรง

ตัวอย่างโค้ด 14.9: leaky_bucket_shaper.py

```
1  #!/usr/bin/env python3
2  import time
3  import threading
4  import queue
5
6  class LeakyBucket:
7      def __init__(self, drain_rate_pps, bucket_capacity):
8          """
9          drain_rate_pps: (*@\thai\font\small อัตราการปล่อยpacket ออก@*) (packet/sec)
10         bucket_capacity: (*@\thai\font\small ขนาดbuffer สูงสุด@*) (packet)
11         """
12         self.drain_rate = drain_rate_pps
13         self.capacity = bucket_capacity
14         self.buffer = queue.Queue(maxsize=bucket_capacity)
15         self.dropped = 0
16         self.sent = 0
17         self._running = True
18
19     def add_packet(self, packet):
20         try:
21             self.buffer.put_nowait(packet)
22         except queue.Full:
23             self.dropped += 1
24             print(f"[DROP] Buffer (*@\thai\font\small เต็ม, packet ถูกทิ้ง@*)(total dropped: {self.dropped})")
25
26     def start_draining(self, output_callback):
27         interval = 1.0 / self.drain_rate
28         def drain_loop():
29             while self._running:
30                 try:
31                     packet = self.buffer.get(timeout=interval)
```

```

32         output_callback(packet)
33         self.sent += 1
34     except queue.Empty:
35         pass
36     time.sleep(interval)
37     threading.Thread(target=drain_loop, daemon=True).start()
38
39     def stop(self):
40         self._running = False
41
42
43 if __name__ == '__main__':
44     def output(pkt):
45         print(f"[SEND @ {time.time():.3f}] {pkt}")
46         (*@\thaifont\small \# ค่าตามparameter ของกลุ่ม@*)
47         bucket = LeakyBucket(drain_rate_pps=10, bucket_capacity=20)
48         bucket.start_draining(output)
49         (*@\thaifont\small \# จำลอง@*) bursty input
50         for i in range(50):
51             bucket.add_packet(f"packet-{i}")
52             time.sleep(0.01) (*@\thaifont\small \# burst เร็วกว่าdrain rate มาก@*)
53
54         time.sleep(5)
55         bucket.stop()
56         print(f"\n(*@\thaifont\small สรุป: สำเร็จ{bucket.sent}, ถูกทิ้ง@*) {bucket.dropped}")

```

จุดสังเกต

สังเกตว่า output ของ Leaky Bucket จะมี timestamp ห่างกันคงที่เสมอ (1/drain_rate วินาที) ไม่ว่า input จะ burst มากเพียงใด ต่างจาก Token Bucket ที่ยอมให้ output burst ได้ทันทีหากมี token สะสมอยู่ในถัง

14.9 ส่วนที่ E: วิเคราะห์ผลด้วย Wireshark

แบบฝึกหัด E

เปิดไฟล์ before_shaping.pcap และ after_shaping.pcap ด้วย Wireshark แล้วทำตามขั้นตอน:

1. ไปที่ **Statistics** → **IO Graph**
2. ตั้งค่า Y-axis เป็น Bits/Tick และ interval เป็น 10 ms

3. เปรียบเทียบกราฟทั้งสองไฟล์ — สังเกต pattern ของ throughput
4. บันทึกภาพกราฟทั้งสองไว้ในรายงาน

Metric	Before Shaping	After Shaping
Peak throughput (Mbps)		
Average throughput (Mbps)		
Packet loss (%)		
Max inter-arrival delay (ms)		
Jitter (std. dev of delay)		

ตารางที่ 14.4: ตารางบันทึกผลการวัด (ให้นักศึกษากรอกจาก Wireshark)

วิธีคำนวณ Jitter ใน Wireshark

ใช้ **Statistics** → **Flow Graph** หรือ export timestamp ของแต่ละ packet ด้วย File → Export Packet Dissections → As CSV แล้วคำนวณส่วนเบี่ยงเบนมาตรฐานของ inter-arrival time ด้วย Python/Excel

14.10 ส่วนที่ F: คำถามวิเคราะห์และอภิปราย

1. จากค่า parameter ของกลุ่มตนเอง (bucket_size_kbit และ token_rate_kbit) เมื่อเพิ่มขนาด burst ใน TBF ให้มากขึ้น ผลลัพธ์ที่เห็นใน Wireshark IO Graph เปลี่ยนไปอย่างไร และเพราะเหตุใด
2. หากลดค่า rate ให้ต่ำกว่า input traffic rate จริงมาก ๆ จะเกิดอะไรขึ้นกับ packet loss และ latency สังเกตได้จากที่ใดใน capture file
3. เปรียบเทียบผลลัพธ์ระหว่างกลุ่มที่ได้ algorithm: token_bucket กับกลุ่มที่ได้ algorithm: leaky_bucket — output traffic pattern ต่างกันอย่างไรเมื่อ input เป็น burst เดียวกัน
4. ในการใช้งานจริง เช่น ISP ที่ทำ bandwidth throttling ควรเลือก Token Bucket หรือ Leaky Bucket และเพราะเหตุใด (พิจารณาจาก traffic ประเภท video streaming เทียบกับ file download)
5. อธิบายว่าเหตุใดการเพิ่ม bucket_size มากเกินไปอาจ ทำให้ Token Bucket ไม่ต่างจากไม่มีการ shaping เลย

ข้อควรระวัง

การทดลองนี้ทำใน Mininet ซึ่งเป็น virtual network environment เท่านั้น ห้ามนำเทคนิคการสร้าง traffic แบบ burst จำนวนมากไปทดสอบกับ เครือข่ายจริงหรือเครือข่ายขององค์กรอื่นโดยไม่ได้รับอนุญาต เนื่องจากอาจถูกตีความเป็นการโจมตีแบบ Denial of Service (DoS)

14.11 ส่วนที่ G: สรุปและส่งงาน

สิ่งที่ต้องส่ง

1. ไฟล์ before_shaping.pcap และ after_shaping.pcap
2. ตารางค่า parameter ประจำกลุ่ม (จาก Part การกำหนดค่า Parameter ประจำกลุ่ม)
3. ตารางบันทึกผลการวัด (Part E) พร้อมภาพ IO Graph ทั้งสองไฟล์
4. คำตอบคำถามวิเคราะห์ทั้ง 5 ข้อ (Part F)
5. Source code ที่ แก้ไข แล้ว (topo_bucket.py, generate_burst_traffic.py, และ leaky_bucket_shaper.py หากกลุ่มได้รับ algorithm นี้)

รูปแบบไฟล์ส่งงาน: PDF report + zip ของ source code และ pcap

เกณฑ์การให้คะแนน (ตัวอย่าง)

ส่วนประกอบ	คะแนน (%)
Topology และการตั้งค่า TBF ถูกต้อง	20
Traffic generation ตรงตาม parameter กลุ่ม	20
คุณภาพการวิเคราะห์ pcap (กราฟ, ตาราง)	30
คำตอบคำถามวิเคราะห์ (Part F)	20
ความครบถ้วนของการส่งงาน	10
รวม	100